09. 사용 예시 및 통합 가이드

기본 사용 예시

1. 간단한 Quote 검증

```
use dcap_qvl::collateral::get_collateral_from_pcs;

use dcap_qvl::verify::verify;

use std::time::SystemTime;



#[tokio::main]

async  fn  main() -> Result<(), Box<dyn  std::error::Error>> {

// Quote 파일 읽기

let  quote = std::fs::read("quote.bin")?;

// Intel PCS에서 Collateral 수집

let  collateral = get_collateral_from_pcs(&quote).await?;

// 현재 시간 가져오기

let  now = SystemTime::now()

.duration_since(SystemTime::UNIX_EPOCH)?

.as_secs();

// Quote 검증

let  verified_report = verify(&quote, &collateral, now)?;
```

```
println!("TCB Status: {}", verified_report.status);

println!("Advisory IDs: {:?}", verified_report.advisory_ids);

Ok(())

}
```

2. 커스텀 PCCS 서버 사용

```
use dcap_qvl::collateral::get_collateral;

use dcap_qvl::verify::verify;


#[tokio::main]

async fn main() -> Result<(), Box<dyn std::error::Error>> {

// 환경 변수에서 PCCS URL 가져오기

let pccs_url = std::env::var("PCCS_URL")

.unwrap_or_else(|_| "https://localhost:8081/sgx/certification/v4/".to_string());

let quote = std::fs::read("tdx_quote.bin")?;

// 커스텀 PCCS에서 Collateral 수집

let collateral = get_collateral(&pccs_url, &quote).await?;

let now = SystemTime::now()

.duration_since(SystemTime::UNIX_EPOCH)?

.as_secs();

let verified_report = verify(&quote, &collateral, now)?;

match verified_report.status.as_str() {
```

```
"UpToDate" => println!("✅ Platform is up to date"),
"OutOfDate" => {
println!("⚠️ Platform needs updates");
for advisory_id in &verified_report.advisory_ids {
println!(" - Advisory: {}", advisory_id);
}
}
_ => println!("❓ TCB Status: {}", verified_report.status),
}
Ok(())
}
```

3. 원스톱 검증 함수 사용

```
use dcap_qvl::collateral::get_collateral_and_verify;

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
let quote = std::fs::read("quote.bin")?;
// Collateral 수집과 검증을 한 번에 수행
let verified_report = get_collateral_and_verify(&quote, None).await?;
println!("Verification Result: {:?}", verified_report);
Ok(())
```

```
}
```

Quote 정보 추출 예시

1. Quote 파싱 및 정보 추출

```
use dcap_qvl::quote::Quote;


fn analyze_quote(quote_data: &[u8]) -> Result<(), Box<dyn std::error::Error>>
let quote = Quote::parse(quote_data)?;
// 기본 정보 출력
println!("Quote Version: {}", quote.header.version);
println!("TEE Type: {}", if quote.header.is_sgx() { "SGX" } else { "TDX" });
println!("Attestation Key Type: {}", quote.header.attestation_key_type);
// FMSPC 추출
let fmspc = quote.fmspc()?;
println!("FMSPC: {}", hex::encode(fmspc));
// CA 타입 확인
let ca_type = quote.ca()?;
println!("CA Type: {}", ca_type);
// Report 타입별 정보 추출
match &quote.report {
dcap_qvl::quote::Report::SgxEnclave(report) => {
```

```
println!("SGX Enclave Report:");
println!(" MR_ENCLAVE: {}", hex::encode(report.mr_enclave));
println!(" MR_SIGNER: {}", hex::encode(report.mr_signer));
println!(" ISV_PROD_ID: {}", report.isv_prod_id);
println!(" ISV_SVN: {}", report.isv_svn);
}
dcap_qvl::quote::Report::TD10(report) => {
println!("TDX Report (v1.0):");
println!(" MR_TD: {}", hex::encode(report.mr_td));
println!(" MR_CONFIG_ID: {}", hex::encode(report.mr_config_id));
println!(" MR_OWNER: {}", hex::encode(report.mr_owner));
}
dcap_qvl::quote::Report::TD15(report) => {
println!("TDX Report (v1.5):");
println!(" MR_TD: {}", hex::encode(report.base.mr_td));
println!(" MR_SERVICE_TD: {}", hex::encode(report.mr_service_td));
}
}
Ok(())
}
```

2. 인증서 체인 분석

```rust
use dcap_qvl::quote::Quote;

use dcap_qvl::utils::extract_certs;



fn  analyze_certificate_chain(quote_data: &[u8]) -> Result<(), Box<dyn  std::err

let  quote = Quote::parse(quote_data)?;

// 인증서 체인 추출

let  raw_cert_chain = quote.raw_cert_chain()?;

let  certs = extract_certs(raw_cert_chain)?;

println!("Certificate Chain ({} certificates):", certs.len());

for (i, cert) in  certs.iter().enumerate() {

// X.509 인증서 파싱

let  cert_der: x509_cert::Certificate = der::Decode::from_der(cert)?;

println!("Certificate {}:", i + 1);

println!(" Subject: {}", cert_der.tbs_certificate.subject);

println!(" Issuer: {}", cert_der.tbs_certificate.issuer);

println!(" Serial: {:?}", cert_der.tbs_certificate.serial_number);

// 유효 기간

let  validity = &cert_der.tbs_certificate.validity;

println!(" Valid From: {:?}", validity.not_before);

println!(" Valid To: {:?}", validity.not_after);

}

Ok(())
```

```
}
```

웹 서비스 통합 예시

1. Axum 웹 서버 통합

```
use axum::{
extract::Multipart,
http::StatusCode,
response::Json,
routing::post,
Router,
};
use dcap_qvl::collateral::get_collateral_and_verify;
use serde::{Deserialize, Serialize};
use std::time::SystemTime;

#[derive(Serialize)]
struct VerificationResponse {
success: bool,
tcb_status: Option<String>,
advisory_ids: Option<Vec<String>>,
error: Option<String>,
```

```
}

async fn verify_quote_handler(mut multipart: Multipart) -> Result<Json<Verifi
while let Some(field) = multipart.next_field().await.map_err(|_| StatusCode::BA
if field.name() == Some("quote") {
let quote_data = field.bytes().await.map_err(|_| StatusCode::BAD_REQUEST)?;
match get_collateral_and_verify(&quote_data, None).await {
Ok(verified_report) => {
return Ok(Json(VerificationResponse {
success: true,
tcb_status: Some(verified_report.status),
advisory_ids: Some(verified_report.advisory_ids),
error: None,
}));
}
Err(e) => {
return Ok(Json(VerificationResponse {
success: false,
tcb_status: None,
advisory_ids: None,
error: Some(e.to_string()),
}));
}
```

```
}
}
}
Err(StatusCode::BAD_REQUEST)
}

#[tokio::main]
async fn main() {
let app = Router::new()
.route("/verify", post(verify_quote_handler));
let listener = tokio::net::TcpListener::bind("0.0.0.0:3000").await.unwrap();
axum::serve(listener, app).await.unwrap();
}
```

2. REST API 클라이언트 예시

```
# Quote 파일 업로드 및 검증
curl -X POST \
-F "quote=@quote.bin" \
http://localhost:3000/verify
```

배치 처리 예시

1. 여러 Quote 파일 일괄 처리

```
use dcap_qvl::collateral::get_collateral_and_verify;

use std::path::Path;

use tokio::fs;


async  fn  batch_verify_quotes(quote_dir: &Path) -> Result<(), Box<dyn  std::err

let  mut  entries = fs::read_dir(quote_dir).await?;

while  let Some(entry) = entries.next_entry().await? {

let  path = entry.path();

if  path.extension().and_then(|s| s.to_str()) == Some("bin") {

println!("Processing: {:?}", path);

match fs::read(&path).await {

Ok(quote_data) => {

match  get_collateral_and_verify(&quote_data, None).await {

Ok(verified_report) => {

println!(" ✅ Status: {}", verified_report.status);

if !verified_report.advisory_ids.is_empty() {

println!(" ⚠️ Advisories: {:?}", verified_report.advisory_ids);

}

}

Err(e) => {

println!(" ❌ Error: {}", e);
```

```
}
}
}
Err(e) => {
println!(" ❌ Failed to read file: {}", e);
}
}
}
}
Ok(())
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
let quote_dir = Path::new("./quotes");
batch_verify_quotes(quote_dir).await?;
Ok(())
}
```

캐싱 및 성능 최적화

1. Collateral 캐싱

```rust
use std::collections::HashMap;

use std::sync::Arc;

use tokio::sync::RwLock;

use dcap_qvl::{QuoteCollateralV3, quote::Quote};



#[derive(Clone)]

struct CollateralCache {

cache: Arc<RwLock<HashMap<String, (QuoteCollateralV3, u64)>>>,

ttl_seconds: u64,

}



impl CollateralCache {

fn  new(ttl_seconds: u64) -> Self {

Self {

cache: Arc::new(RwLock::new(HashMap::new())),

ttl_seconds,

}

}

async  fn  get_or_fetch_collateral(

&self,

quote_data: &[u8],

pccs_url: Option<&str>,
```

```
) -> Result<QuoteCollateralV3, Box<dyn  std::error::Error>> {

let  quote = Quote::parse(quote_data)?;

let  fmspc = hex::encode(quote.fmspc()?);

let  ca = quote.ca()?;

let  cache_key = format!("{}_{}", fmspc, ca);

let  now = std::time::SystemTime::now()

.duration_since(std::time::UNIX_EPOCH)?

.as_secs();

// 캐시에서 확인

{

let  cache = self.cache.read().await;

if  let Some((collateral, timestamp)) = cache.get(&cache_key) {

if  now - timestamp < self.ttl_seconds {

return Ok(collateral.clone());

}

}

}

// 캐시 미스 또는 만료 - 새로 가져오기

let  collateral = if  let Some(url) = pccs_url {

dcap_qvl::collateral::get_collateral(url, quote_data).await?

} else {

dcap_qvl::collateral::get_collateral_from_pcs(quote_data).await?

};
```

```rust
// 캐시 업데이트
{
let mut cache = self.cache.write().await;
cache.insert(cache_key, (collateral.clone(), now));
}
Ok(collateral)
}
}

// 사용 예시
#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
let cache = CollateralCache::new(3600); // 1시간 TTL
let quote = std::fs::read("quote.bin")?;
let collateral = cache.get_or_fetch_collateral(&quote, None).await?;
let now = std::time::SystemTime::now()
.duration_since(std::time::UNIX_EPOCH)?
.as_secs();
let verified_report = dcap_qvl::verify::verify(&quote, &collateral, now)?;
println!("Status: {}", verified_report.status);
Ok(())
}
```

에러 처리 및 로깅

1. 구조화된 로깅

```
use tracing::{info, warn, error, instrument};

use dcap_qvl::collateral::get_collateral_and_verify;



#[instrument(skip(quote_data))]

async  fn  verify_with_logging(

quote_data: &[u8],

pccs_url: Option<&str>,

) -> Result<dcap_qvl::verify::VerifiedReport, Box<dyn  std::error::Error>> {

info!("Starting quote verification, size: {} bytes", quote_data.len());

match  get_collateral_and_verify(quote_data, pccs_url).await {

Ok(verified_report) => {

info!(

tcb_status = %verified_report.status,

advisory_count = verified_report.advisory_ids.len(),

"Quote verification successful"

);

if !verified_report.advisory_ids.is_empty() {

warn!(

advisory_ids = ?verified_report.advisory_ids,
```

```
"Security advisories found"
);
}
Ok(verified_report)
}
Err(e) => {
error!(error = %e, "Quote verification failed");
Err(e)
}
}
}
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
tracing_subscriber::fmt::init();
let quote = std::fs::read("quote.bin")?;
let _result = verify_with_logging(&quote, None).await?;
Ok(())
}
```

테스트 및 검증

1. 단위 테스트 예시

```rust
#[cfg(test)]
mod tests {
use super::*;
use dcap_qvl::quote::Quote;
#[test]
fn test_quote_parsing() {
let quote_data = include_bytes!("../test_data/sgx_quote.bin");
let quote = Quote::parse(quote_data).expect("Failed to parse quote");
assert_eq!(quote.header.version, 3);
assert!(quote.header.is_sgx());
assert_eq!(quote.header.attestation_key_type, 2);
}
#[tokio::test]
async fn test_quote_verification() {
let quote_data = include_bytes!("../test_data/valid_quote.bin");
let result = get_collateral_and_verify(quote_data, None).await;
assert!(result.is_ok());
let verified_report = result.unwrap();
assert!(!verified_report.status.is_empty());
}
}
```

2. 통합 테스트

```rust
use std::process::Command;


#[tokio::test]
async fn integration_test_with_real_quote() {
// 실제 TEE 환경에서 Quote 생성 (예시)
let output = Command::new("sgx_quote_generator")
.arg("--output")
.arg("test_quote.bin")
.output()
.expect("Failed to generate quote");
assert!(output.status.success());
// 생성된 Quote 검증
let quote_data = std::fs::read("test_quote.bin").unwrap();
let result = get_collateral_and_verify(&quote_data, None).await;
assert!(result.is_ok());
// 정리
std::fs::remove_file("test_quote.bin").ok();
}
```

이러한 예시들을 통해 다양한 환경과 요구사항에 맞게 DCAP Quote 검증 기능을 통합할 수 있습니다.